

Chapter 11. Noncompliance and Instruments

It's not uncommon for companies to offer products or services to their existing customer base. For instance, a retailer can offer a subscription-based program where customers get free shipping. A streaming company can offer an ad-free version of its services for an additional fee. Or a bank can offer a prime credit card with lots of perks for customers who spend above a certain threshold.

In all of these examples, *the customer must opt in* for the additional service, which makes inferring its impact challenging. As the choice to participate lies with the customer, that choice often confounds the impact evaluation of the service; after all, customers who opt in and customers who don't will likely have different Y_0 . Even if the company *randomizes the availability* of the service or product, it can't force customers to take it. This is called *noncompliance*, where *not everyone that gets assigned to the treatment takes it*. This chapter will walk you through how to think about this issue and what to consider when you want to design an experiment that suffers from noncompliance.

Noncompliance

Noncompliance comes from pharmaceutical science (although some of the tooling to deal with it comes from economics).

Imagine you are conducting an experiment to test the effect of a new drug on some illness. Each subject gets assigned to a treatment: a drug or a placebo. But those subjects are imperfect human beings, who sometimes forget to take their medicine. As a result, not everyone assigned to the treatment gets it. Also, someone critically ill might figure out they were assigned the placebo and manage to find a way to get the treatment regardless. That is to say, if you *separate the treatment assignment from the treatment intake*, you'll end up with four groups:

Compliers

Those who take the treatment that was assigned to them

Always takers

Those who always take the treatment, regardless of the assignment

Never takers

Those who never take the treatment, regardless of the assignment

Defiers

Those who take the opposite treatment from the one assigned

The catch here is that you don't know who belongs to each group.

You can also represent noncompliance in a DAG (see [Figure 11-1](#)), where Z is the treatment assignment (random in this case), T is the treatment, Y is the outcome, and U is hidden factors that confound the treatment choice and the outcome. Z is what is called *an instrument*: a variable that (1) impacts the treatment in a nonconfounded way and (2) doesn't impact the outcome, unless through the treatment.

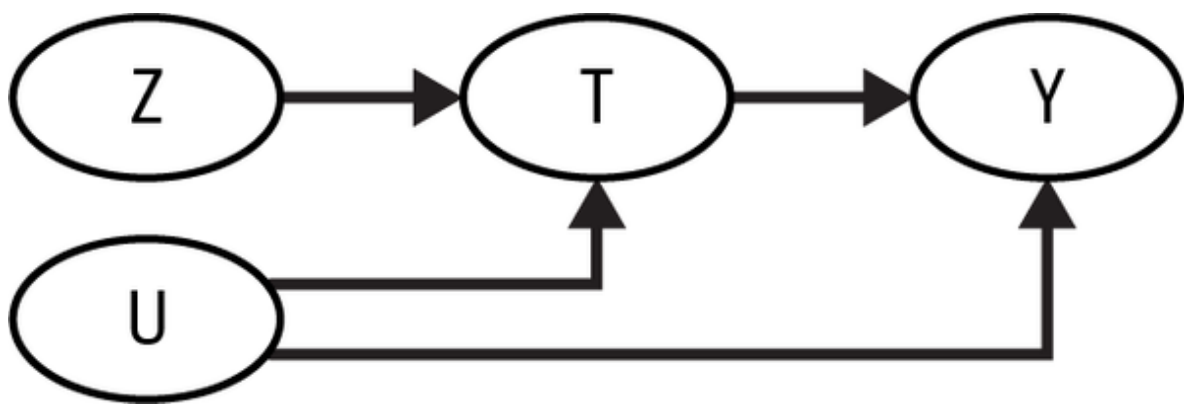


Figure 11-1. The canonical instrumental variable DAG

Since the compliance group and the treatment assignment deterministically cause the treatment intake, you can also think of U as unknown factors that cause the compliance group. As you can see from the DAG in [Figure 11-1](#), without further assumptions, you can't identify the effect of the treatment on Y due to an open backdoor path through U . As you'll soon see, identification of that effect will involve clever usage of Z .

To make things more concrete, let's work with a typical industry setting where a bank wants to know the impact of offering a prime credit card to its customers. Since this prime

service is costly, the bank charges a small fee, which is not enough to cover all its costs. But, if the purchase volume—the total amount spent on the card—of those prime customers increases by at least 500 USD, then it is worth it. Hence, the bank wants to know how much the prime card increases customers' purchase volume.

The bank managed to run an experiment where it randomized the availability of the prime credit card

(`prime_eligible`) to 10,000 customers, with each customer having a 50% chance of being eligible and 50% of being in the control. Of course, the bank can't force customers to choose the card, making this an experiment with non-compliance.

If you map these variables to [Figure 11-1](#), purchase volume would be Y , availability of the prime credit card would be Z , and having the prime card would be T . All this information is stored in the following data frame. The bank also has information on the customer's age, income, and credit score, but let's not worry about those variables for now. Additionally, I've added information on the true effect of the prime card on PV, τ (`tau`), and which group a customer belongs to. Keep in mind that the compliance category and τ are not available to you in real life. I'll only use them here to make some examples easier to understand:

```
In [1]: import pandas as pd
import numpy as np

df = pd.read_csv("../data/prime_card.csv")

df.head()
```

	age	income	credit_score
0	37.7	9687.0	822.0
1	46.0	13731.0	190.0
2	43.1	2839.0	214.0
3	36.0	1206.0	318.0
4	39.7	4095.0	430.0

Extending Potential Outcomes

To be more precise with noncompliance and to proceed with identification, you'll have to extend the potential outcome

notation. Since Z causes T , you can now define a *potential treatment* T_z . Also, the potential outcome has new counterfactuals with respect to the instrument Z , $Y_{z,t}$.

In the prime credit card example, Z is randomized, which means that the effect of Z on Y —also called the *intention-to-treat effect* (ITTE)—is pretty easy to identify:

$$ITTE = E[Y|Z = 1] - E[Y|Z = 0] = E[Y_{1,t} - Y_{0,t}],$$

which you can estimate with a simple linear regression:

```
In [2]: m = smf.ols("pv~prime_eligible", data=d)
        m.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	2498.3618	24.327	102.701	0.000
prime_eligible	321.3880	34.321	9.364	0.000

The ITTE is a valuable metric in its own right as it measures the impact of assigning a treatment, such as offering the prime credit card in this scenario. For the bank, this number indicates the additional purchase volume (PV) per customer it can expect by having the prime credit card available as part of its product suite. However, it's crucial to note that the ITTE is not the same as the treatment effect. The bank's primary

objective is to determine whether the benefits of the prime card outweigh its costs. Therefore, the bank needs to identify the treatment effect of choosing the card, rather than solely relying on the ITTE.

In this particular example, the bank has full control on who has the prime card available. As a result, you have *one-sided noncompliance*, since there is no way for customers who are not eligible for the prime card to get it, but customers who have the card available can still choose to not have it. This forces the always takers into compliance and the defiers into never takers, reducing the number of compliance groups from four to two.

Now that you understand the setting, let's think about identifying the effect of the prime card. An obvious idea is to use the ITTE as a proxy for the card effect. Maybe they are not so different after all. So, what is the ITTE anyway?

Due to randomization of treatment assignment, the ITTE can be obtained by comparing those assigned to the treatment to those assigned in the control. But you can quickly see that this comparison will give you a *biased toward zero estimate for the treatment effect* (see [Figure 11-2](#)). That's because some of those assigned to the treatment actually get the control, which decreases the perceived difference between the two groups.

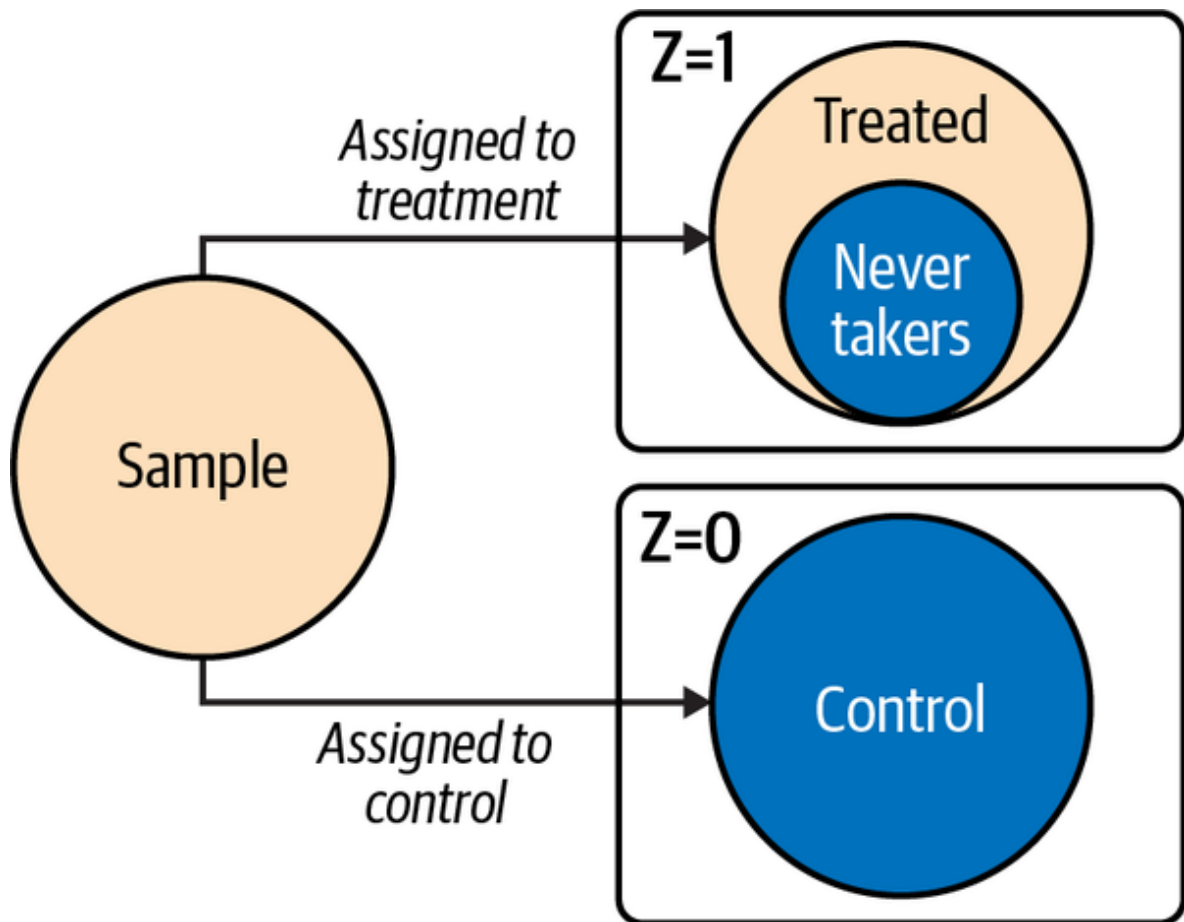


Figure 11-2. The ITTE is a biased toward zero estimate of the ATE, since some of the units assigned to one treatment actually get the other treatment

To prove that, you can take advantage of those `tau`s I've added to the dataset. The average treatment effect is quite larger than the ITTE:

```
In [3]: df["tau"].mean()
```

```
Out[3]: 413.45
```


OK, so that turned out to be a dead end. But what about a simple average comparison between treated and control $E[Y|T = 1] - E[Y|T = 0]$? Maybe the randomization assignment will make that a good proxy for the effect estimate you care about. Well... let's estimate that and see:

```
In [4]: m = smf.ols("pv~prime_card", data=df).fit()
        m.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	2534.4947	19.239	131.740	0.000
prime_card	588.1388	41.676	14.112	0.000

Now the measured effect is much larger than the true effect (see [Figure 11-3](#)). The reason is that, in this example the bias is upward, $E[Y_0|T = 1] > E[Y_0|T = 0]$, meaning that customers who choose the prime card spend more regardless of the prime card. In other words, the never takers have lower Y_0 than the compliers, which artificially lowers the average outcome of the untreated group.

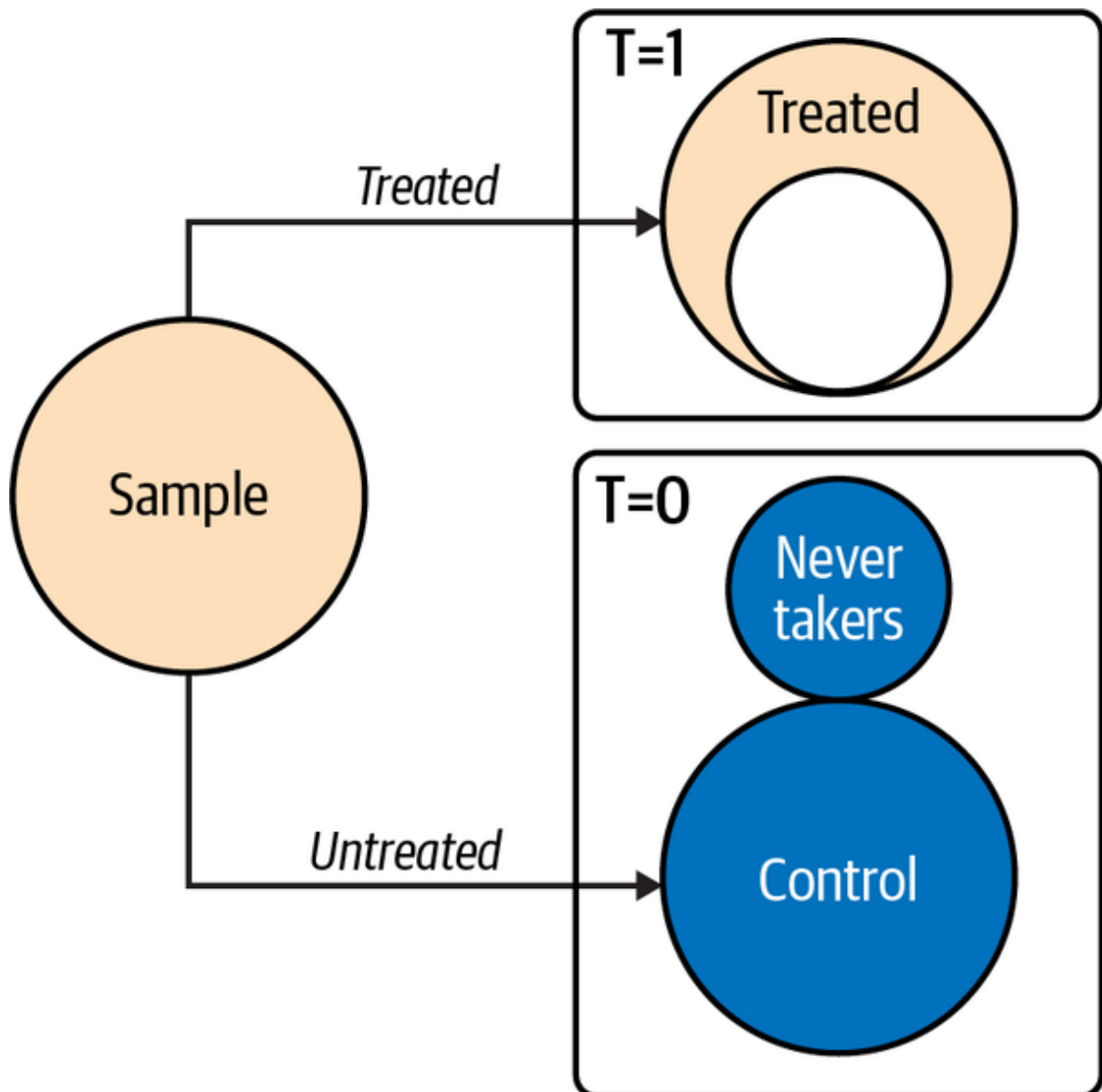


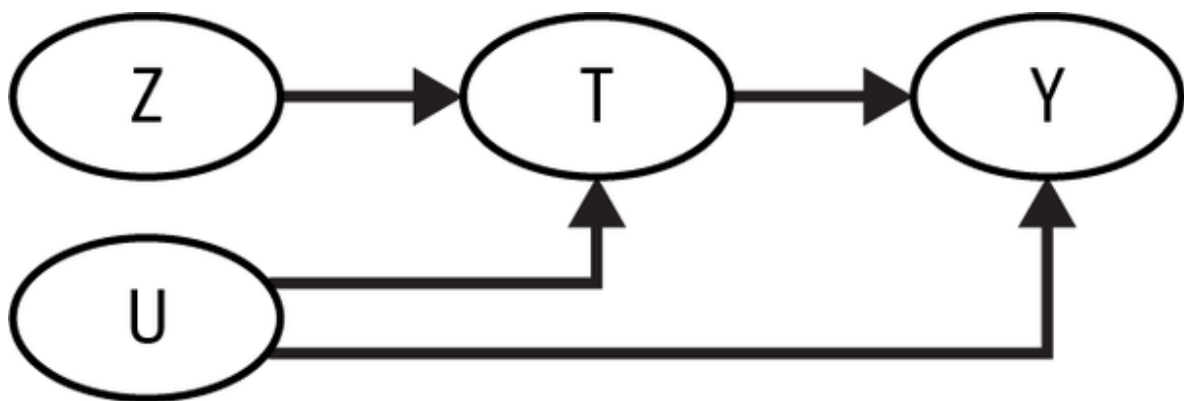
Figure 11-3. Under noncompliance, comparing treated and untreated will not recover the ATE, since treatment choice is not random

It seems that you are a bit stuck here. The ATE can't be identified and ITTE is a biased measure for it. As with much in causal inference, in order to make some progress, you'll need to make additional assumptions.

Instrument Identification

Assumptions

Let's take the DAG I showed you earlier and reproduce it here for better readability. As you'll see, the first few assumptions you'll need for identification are already spelled out in that DAG:



Here they are:

1. The first assumption you need is *independence*; the lack of unmeasured confounders between Z and T , $T_z \perp Z|X$, and between Z and Y , $Y(Z, T_z) \perp Z|X$. This assumption states that the instrument is as good as randomly assigned. This assumption is not testable, but it can be made more plausible by the experiment design. In your example, you can probably say this assumption is satisfied, since the bank randomized the availability of the prime card.

2. The second assumption is the *exclusion restriction*, $Y_{z,t} = Y_t$, which is the lack of a path from Z to Y that does not go through the treatment T . In words, it says that the instrument only affects the outcome through the treatment. This one is more tricky. Even if Z is randomized, it could impact the outcome through other channels. For example, let's say that customers figured out which group they were assigned to and those in the control got very mad at the bank and decided to close their accounts. In this case, randomization affects the outcome through a channel that is not the treatment.
3. The third assumption is *relevance*, $E[T_1 - T_0] \neq 0$, which is the existence of an arrow from Z to T . This assumption states that the instrument must have an influence on the treatment. Fortunately, this assumption is testable, since you can estimate the effect of the instrument on the treatment.
4. The fourth and final assumption is not stated in the DAG. It is mostly a functional form assumption imposed on the causal model: *monotonicity*, $T_{i1} \geq T_{i0}$ (or vice versa). It might look confusing, but it simply states that the instrument flips the treatment in only one direction. It either increases the chance of getting the treatment for everyone who got the instrument, which is equivalent to assuming that there are no defiers; or it decreases that chance, which is equivalent to assuming that there are no

compliers. In your example, this is also a plausible assumption, as customers in the control group can't force their way into getting the prime credit card. As a result, the defiers are collapsed into the never takers.

Now, let's see how to use those assumptions for identification. The goal here is to start with the ITTE and see if we can get something like an average treatment effect. First, let's expand the outcome into the potential outcomes. Recall that you can do this using the treatment as a switch—

$$Y = Y_1T + Y_0(1 - T):$$

$$\begin{aligned} E[Y|Z = 1] - E[Y|Z = 0] &= E[Y_{1,1}T_1 + Y_{1,0}(1 - T_1)|Z = 1] \\ &\quad - E[Y_{0,1}T_0 + Y_{0,0}(1 - T_0)|Z = 0] \end{aligned}$$

Now, because of the exclusion restriction, you can remove the instrument subscript of $Y_{z,t}$:

$$E[Y_1T_1 + Y_0(1 - T_1)|Z = 1] - E[Y_1T_0 + Y_0(1 - T_0)|Z = 0]$$

Next, using the independence assumption, you can merge both expectations:

$$E[Y_1T_1 + Y_0(1 - T_1) - Y_1T_0 - Y_0(1 - T_0)]$$

which you can simplify to

$$E[(Y_1 - Y_0)(T_1 - T_0)].$$

Next, let's use the monotonicity assumption and expand this expectation into the possible cases, $T_1 > T_0$ and $T_1 = T_0$:

$$\begin{aligned}
& E[(Y_1 - Y_0)(T_1 - T_0)|T_1 > T_0] * P(T_1 > T_0) \\
+ & E[(Y_1 - Y_0)(T_1 - T_0)|T_1 = T_0] * P(T_1 = T_0)
\end{aligned}$$

And since $T_1 - T_0$ would be 0 if $T_1 = T_0$, you are left with only the first term. Since Z is binary, $T_1 - T_0 = 1$ and:

$$E[Y|Z = 1] - E[Y|Z = 0] = E[Y_1 - Y_0|T_1 > T_0] * P(T_1 > T_0).$$

This is a good time to pause and see what you've accomplished. First, notice that $T_1 > T_0$ are the compliers, the population where the instrument shifts the treatment from 0 to 1. This means that this last equation tells you that *the effect of the instrument on the outcome is the treatment effect of the compliers times the compliance rate*. This explains why the ITTE is a biased-toward-zero estimate for this effect: you are multiplying it by a rate, which is between 0 and 1. If you could only estimate $P(T_1 > T_0)$, then you would be able to correct the previous estimator...

But wait a second! Since the instrument is randomized, you can estimate its impact on the treatment, $E[T_1 - T_0]$. And since $T_1 - T_0 = 1$ for the compliers and 0 otherwise (due to the monotonicity assumption), this effect is the compliance rate:

$$E[T_1 - T_0] = P(T_1 > T_0)$$

Putting it all together, this means that you can *identify the average treatment effect on the compliers by scaling up the effect of the instrument on the outcome by the compliance rate*, which is the effect of the instrument on the treatment:

$$E[Y_1 - Y_0 | T_1 > T_0] = \frac{E[Y|Z = 1] - E[Y|Z = 0]}{E[T|Z = 1] - E[T|Z = 0]}$$

This is how you can use instruments to identify the effect in noncompliance settings. The good news is that you can identify that effect. The bad news is that it is not the ATE, but only the effect on the compliers, which is usually called the *local average treatment effect* (LATE). Unfortunately, you can't identify the ATE. But this might not be a problem. In your credit card example, the LATE would be the effect on those who choose the prime card when it is available to them. Now, the bank wants to know if the effect in terms of extra *PV* compensate for the cost of the prime card, both of which only occur for those who choose the prime card. In this situation, it's enough to know the LATE. The bank doesn't care about the effect of those who will never opt in for the prime card.

Having gone through the theory about instrument identification, it is now time to check how to apply it in practice.

First Stage

The first step in instrumental variables analysis is to run what is conveniently called a *first stage* regression, where you regress the treatment on the instrument. During this step, you can check the relevance assumption—if the parameter estimate associated with the instrument is large and

statistically significant, you have good reason to believe that assumption holds:

```
In [5]: first_stage = smf.ols("prime_card ~ prime_eligible")
first_stage.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	6.729e-15	0.005	1.35e-12	1.000
prime_eligible	0.4242	0.007	60.536	0.000

In this example, the compliance rate is estimated to be about 42%, which is also statistically significant (the 95% CI is [0.410, 0.438]). Since I've included the true group to which each customer belongs, you can even check if that is indeed the actual compliance rate:

```
In [6]: df.groupby("categ").size() / len(df)
```

```
Out[6]: categ
complier      0.4269
never-taker   0.5731
dtype: float64
```


Reduced Form

The second step is called the *reduced form*. At this stage, you regress the outcome on the instrument to obtain the intention to treat the effect:

```
In [7]: red_form = smf.ols("pv ~ prime_eligible")  
red_form.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	2498.3618	24.327	102.701	0.000
prime_eligible	321.3880	34.321	9.364	0.000

Once you've run the first stage and the reduced form, you can divide the parameter estimate from the first by the parameter estimate on the latter to obtain the estimate for the local average treatment effect:

```
In [8]: late = (red_form.params["prime_eligible"]  
               first_stage.params["prime_eligible"]  
               )  
late
```

```
Out[8]: 757.6973795343938
```

As you can see, this effect is more than twice the ITTE. This is expected, since the compliance rate is lower than 50%. It is also larger than the ATE, but that is because the compliers have a higher than average effect. In fact, if you compute the effect for the compliers, you can see that your LATE estimate is pretty close to it:

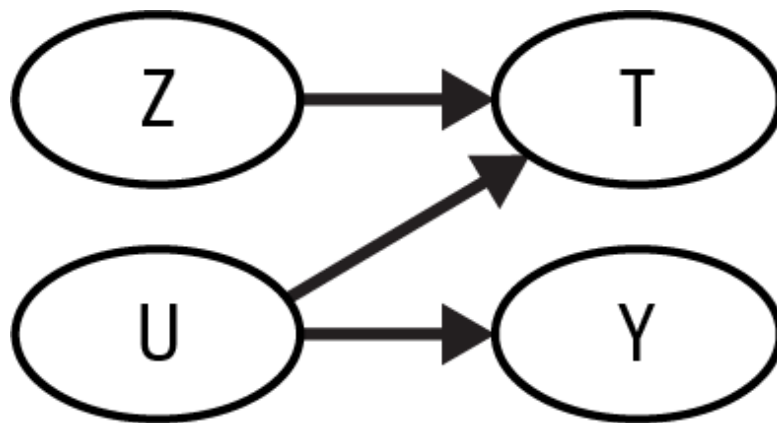
```
In [9]: df.groupby("categ")["tau"].mean()
```

```
Out[9]: categ
complier      700.0
never-taker    200.0
Name: tau, dtype: float64
```

There is still some difference, though. It's hard to tell if this is right or not if you don't wrap that point estimate in a confidence interval. You could do that using bootstrap, but I think it is worth looking into the actual formula for the standard error of instrumental variable (IV) estimates. To do that, you have to learn an alternate way to estimate the LATE.

Two-Stage Least Squares

If you zoom in the treatment part of the DAG, you can see that it is caused by two components: first, there is a random component, which is the randomized instrument. Second, there is the U component, which is where the confounding bias comes from:



Recall how the first stage is a regression of the treatment on the instrument, which essentially means you are estimating the path $Z \rightarrow T$. But there is more to it. Since that is what the first stage is estimating, you can think of its predicted values, $\hat{T}$, as an unbiased version of the treatment. Which in turn means that if you regress the outcome on those predicted values, you'll get the same IV estimate as before:

```
In [10]: iv_regr = smf.ols(  
          "pv ~ prime_card",  
          data=df.assign(prime_card=first_sta
```

```
iv_regr.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	2498.3618	24.327	102.701	0.000
prime_card	757.6974	80.914	9.364	0.000

This approach is called two-stage least squares (2SLS). OK, but why is it useful? First, because it will allow you to properly compute standard errors; second, because it makes adding more instruments and covariates as easy as adding variables in a regression model. Let's now talk about each of those in turn.

Standard Error

The residuals from the second stage's prediction can be defined as follows:

$$\hat{e}_{IV} = Y - \hat{\beta}_{IV}T$$

Notice that this is *not* the same residuals you would get with the `.resid` method from the second stage, since those would be $Y - \hat{\beta}_{IV}\hat{T}$. The residual you want uses the raw version of the treatment, not the predicted one.

With that residual, you can compute the standard error for the IV estimates:

$$SE(\hat{\beta}_{IV}) = \frac{\sigma(\hat{\epsilon}_{IV})}{\hat{\beta}_{z,1st}\sigma(Z)\sqrt{n}},$$

where $\sigma(\cdot)$ stands for the standard deviation function and $\hat{\beta}_{z,1st}$ is the estimated compliance rate, which you got from the first stage:

```
In [11]: Z = df["prime_eligible"]
         T = df["prime_card"]
         n = len(df)

         # not the same as iv_regr.resid!
         e_iv = df["pv"] - iv_regr.predict(df)
         compliance = np.cov(T, Z)[0, 1]/Z.var()

         se = np.std(e_iv)/(compliance*np.std(Z))

         print("SE IV:", se)
         print("95% CI:", [late - 2*se, late + 2*se])
```

```
Out[11]: SE IV: 80.52861026141942
         95% CI: [596.6401590115549, 918.7546000115549]
```

Just to double-check your results, you can use the `2SLS` module from the `linearmodels` Python package. With it, you can wrap the first stage, as in `[T~Z]`, inside the model's formula and fit an IV model. As you can see, using this package gives not only the same LATE estimate as the one you got earlier, but also the same standard error as the one you've just calculated:

```
In [12]: from linearmodels import IV2SLS

        formula = 'pv ~ 1 + [prime_card ~ prime_card]'
        iv_model = IV2SLS.from_formula(formula, data)

        iv_model.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value
Intercept	2498.4	24.211	103.19	0.0000
prime_card	757.70	80.529	9.4090	0.0000

Regardless of the method you use, you can see that this is a pretty huge confidence interval, even if it does contain the true ATE for the compliers, which is 700. More importantly, I think that the standard error formula can shed some light on the challenges of noncompliance experiments. First, notice $\sigma(Z)$ in the denominator. Since Z is a binary variable, the

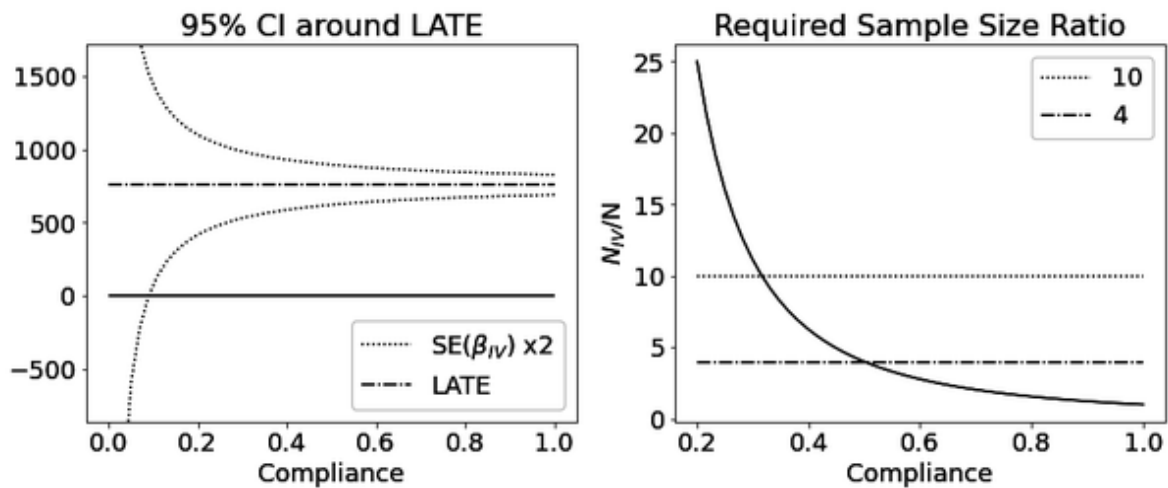
maximum value for $\sigma(Z)$ is 0.5. This is not much different from OLS with a binary treatment. (Recall that then, the standard error was $\sigma(\hat{\epsilon}) / (\sigma(T)\sqrt{n})$). It simply states that you can maximize the power of a test by randomizing the treatment in a 50%-50% fashion.

But now you have an extra term in the denominator: the compliance rate, $\hat{\beta}_{z,1st}$. Not surprisingly, if compliance is 100%, then $Z = T$, $\hat{\beta}_{z,1st} = 1$ and you get back the OLS standard error. But with noncompliance, the standard error increases, since $\hat{\beta}_{z,1st} < 1$. For instance, with 50% compliance, the IV standard error will be twice as large as the OLS standard error. As a result, the required sample size for an experiment with 50% compliance is 4x the sample you would need if you had 100% compliance.

BIAS OF IV

It turns out that the IV estimates are consistent, but not unbiased. That is, $E[\beta_{IV}] \neq \beta$. This is mostly due to sampling error in the first stage. Since you don't have infinite data, the fitted value for the treatment, $\hat{T}$, will be a function of both Z and U , meaning that not all the bias will go away. As you gather more data $\hat{T}$ will become less and less a function of U . This is why the estimator is consistent, meaning that $\text{plim}_{n \rightarrow \infty} \beta_{IV} = \beta$.

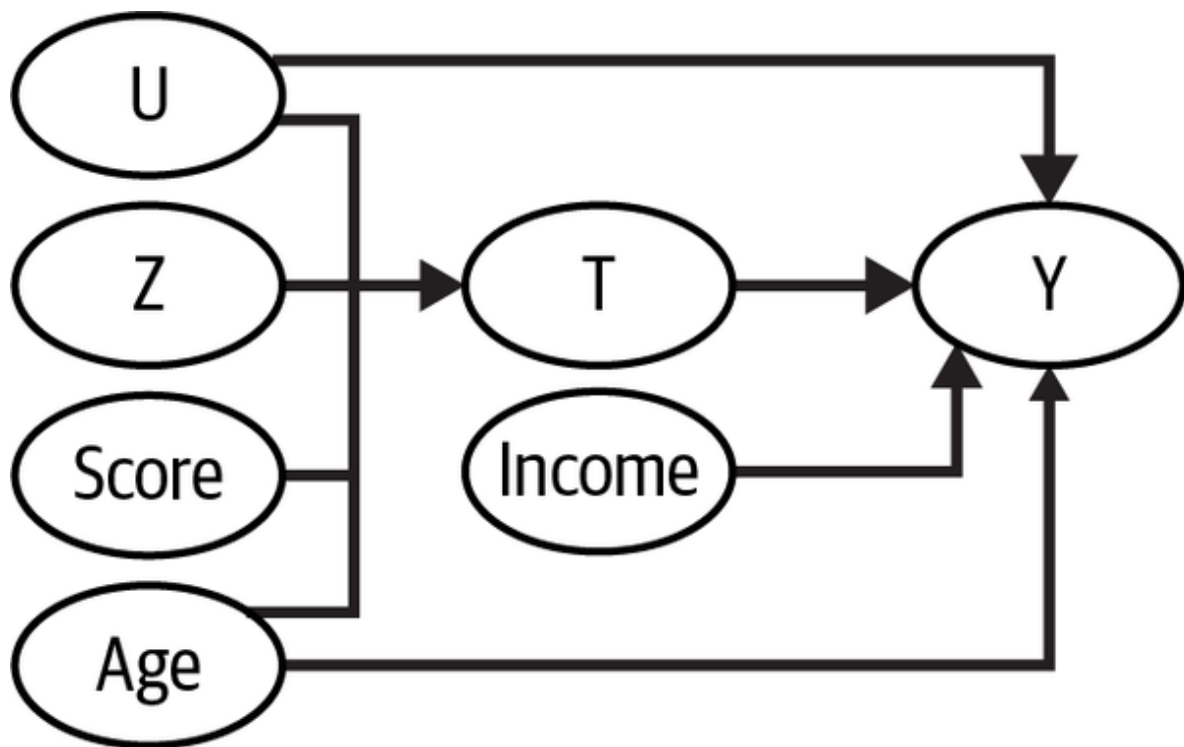
The following plot compares the size of the confidence interval for your LATE parameter estimate assuming different estimated compliance rates (first image). It also shows how many more samples you would need for a test with noncompliance, considering multiple compliance rates:



Fifty percent compliance is still a lot. In most applications, only a small fraction of customers opt in for the prime service or product, which makes it even harder to estimate the LATE. For instance, if compliance is as low as 30%, you'll need a sample 10x larger than the one you would need if compliance weren't an issue. Gathering a sample that big tends to be impractical, if not impossible. But if you run into a problem like this, not all is lost. There are still some tricks you can use to lower the IV standard error. In order to do that, you'll have to include extra covariates in your analysis.

Additional Controls and Instruments

Remember how the prime credit card data had three additional covariates, besides the treatment, the instrument, and the outcome? They were the customer's income, age, and credit score. Now, suppose that the causal graph that describes their relationship with T , Z , and Y is as follows:



That is, income is highly predictive of the outcome, but doesn't predict compliance; credit score predicts compliance, but not the outcome, and age predicts both of them (is a confounder). If you are smart about how you use those variables, you can decrease the standard error by using all of them.

First, let's look at credit score. Credit score causes compliance, but does not cause the outcome. This means that it can be treated as an additional instrumental variable. From the DAG, you can see that it satisfies the first three IV assumptions, just like *Z*. You only have to assume positivity. Including that variable as an extra IV in your 2SLS model will significantly reduce the standard error of the LATE parameter:

```
In [13]: formula = 'pv ~ 1 + [prime_card ~ prime_card]
iv_model = IV2SLS.from_formula(formula,
                                exogenous='prime_card',
                                endogenous='pv',
                                model_type='2SLS')
iv_model.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value
Intercept	2519.4	21.168	119.02	0.0000
prime_card	659.04	58.089	11.345	0.0000

Now, you have to be careful here. If instead of treating it as an instrument, you condition on it, adding it to the second stage too, the error will increase. But you know this already, from [Chapter 4](#), where you learned that conditioning on variables that cause the treatment, but not the outcome, will increase variance. The bigger issue here is that, unless you know the instrument assignment mechanism (as you do with Z), it is hard to know if the exclusion restriction holds. For instance, you are assuming that credit score doesn't cause the outcome mostly because I told you so and you trust me since I was the one who generated the data. But in real life, it's hard to find instruments like that. Most likely, a covariate affects both compliance and the outcome, which is the case of age here. For this reason, a much more promising approach to reducing variance of the IV estimate is to include controls that are

highly predictive of the outcome. For instance, in this example, customer income is very predictive of purchase volume, so including it as an additional control will lower the standard error quite substantially:

```
In [14]: formula = '''pv ~ 1
          + [prime_card ~ prime_eligible + credit
          + income + age]'''

iv_model = IV2SLS.from_formula(formula,

iv_model.summary.tables[1]
```

	Parameter	Std. Err.	T-stat	P-value
Intercept	210.62	37.605	5.6008	0.0000
age	9.7444	0.8873	10.982	0.0000
income	0.3998	0.0008	471.04	0.0000
prime_card	693.12	12.165	56.978	0.0000

As for variables like age, which affect both the outcome and compliance, the effect on the standard error will be more nuanced. Like with the regression case, if it explains the treatment a lot more than the outcome, it might end up increasing the variance.

2SLS by Hand

Since you won't always have instrumental variable software at your disposal, I think it is worth learning how to implement 2SLS by hand, especially since it is not at all complicated. If you have more than one instrument and additional covariates, you can include them in your model by:

1. Running the first stage, regressing the treatment on

instruments and the additional covariates, $T \sim Z + X$.

2. Running the second stage by regressing the outcome on the treated fitted values (from the first stage) and the additional covariates, $Y \sim T_hat + X$:

```
In [15]: formula_1st = "prime_card ~ prime_eligible"
         first_stage = smf.ols(formula_1st, data=df)

         iv_model = smf.ols(
             "pv ~ prime_card + income + age",
             data=df.assign(prime_card=first_stage.fittedvalues),
             covariates=["income", "age"])

         iv_model.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	210.6177	40.832	5.158	0.000
prime_card	693.1207	13.209	52.474	0.000
income	0.3998	0.001	433.806	0.000
age	9.7444	0.963	10.114	0.000

Matrix Implementation

This will give you the exact same IV estimate as the one you got with `linearmodels`, but the standard errors will be off. If you want those, you are probably better off with the matrix implementation of 2SLS. To do that, you have to append the additional covariates into both the treatment and the instrument matrix. Then, you can find the IV parameters as follows:

$$\hat{X} = Z(Z'Z)^{-1}Z'X$$

$$\beta_{IV} = \left(\hat{X}'\hat{X} \right)^{-1} \hat{X}'Y$$

When coding it up, you just have to be careful with large N . $Z(Z'Z)^{-1}Z'$ will be an huge $N \times N$ matrix, which can be avoided if you just first multiply $(Z'Z)^{-1}Z'X$ and then pre-multiply Z :

```
In [16]: Z = df[["prime_eligible", "credit_score", "age"]]
         X = df[["prime_card", "income", "age"]]
```

```

Y = df[["pv"]].values

def add_intercept(x):
    return np.concatenate([np.ones((x.shape[0], 1)), x])

Z_ = add_intercept(Z)
X_ = add_intercept(X)

# pre-multiplying Z_.dot(...) last is faster
# creating a huge NxN matrix
X_hat = Z_.dot(np.linalg.inv(Z_.T.dot(Z_)))

b_iv = np.linalg.inv(X_hat.T.dot(X_hat)).dot(X_hat.T.dot(Y))
b_iv[1]

```

```
Out[16]: array([693.12072518])
```

Once more, you have the exact same coefficient as before.
Once you have that, you can compute the IV residuals and the variance:

$$\widehat{Var}(\hat{\beta}_{IV}) = \sigma^2(\hat{e}_{iv}) \text{diag} \left(\left(\hat{X}' \hat{X} \right)^{-1} \right)$$

```

In [17]: e_hat_iv = (Y - X_.dot(b_iv))

var = e_hat_iv.var()*np.diag(np.linalg

```

```
np.sqrt(var[1])
```

```
Out[17]: 12.164694395033125
```

This variance formula is a bit harder to interpret, due to the matrix notation, but you can approximate something that is more in line with what you had before, in the case without additional covariates:

$$SE(\widehat{\beta}_{IV}) \approx \frac{\sigma(\hat{e}_{IV})}{\sigma(\tilde{T})\sqrt{nR_{1st}^2}}$$

Here, $\tilde{T}$ is the residuals of the treatment regressed on the additional covariates, but not the instrument, and R_{1st}^2 is the R^2 from the first stage:

```
In [18]: t_tilde = smf.ols("prime_card ~ income  
e_hat_iv.std() / (t_tilde.std() * np.sqrt(n * R_1st_sq))
```

```
Out[18]: 12.156252763192523
```

With this formula, you can see that, aside from increasing the sample size, you have three levers to decrease the standard error:

1. Increasing the first stage R^2 . This can be done by finding strong instruments, which are variables that are good at predicting compliance, but satisfy the exclusion restriction (do not cause the outcome).
2. Removing variables that are highly predictive of T in order to increase $\sigma(\tilde{T})$.
3. Decreasing the size of the second stage residuals, which can be done by finding variables that are highly predictive of the outcome.

Of those three levers, I confess I only like the last one. As I said before, it is very hard to find IVs in the wild. Also, there is only so much you can remove in order to decrease $\sigma(\tilde{T})$. Which leaves you with the only reliable way to decrease the variance: finding variables that are good at predicting the outcome.

Discontinuity Design

Regression discontinuity design (RDD) is another design worth mentioning in addition to the traditional instrumental variable and noncompliance designs. Although RDD is widely used in academia, its application in industry may be more

limited. RDD leverages artificial discontinuities in the treatment assignment to identify the treatment effect. For example, suppose a government implements a money transfer program that offers poor families a monthly check of 200 USD in the local currency, but only families earning less than 50 USD are eligible. This creates a discontinuity in the program's assignment at 50 USD, allowing researchers to compare families just above and just below the threshold to measure the program's effectiveness, provided that the two groups are similar.

RDD can be applied to many other situations besides the money transfer program example. Discontinuities are pervasive, making RDD very attractive to researchers. For instance, to understand the impact of college, researchers can compare people who scored just above and just below a passing threshold in an admission exam. To assess the impact of women on politics, researchers can compare cities where a female candidate lost by a small margin to those where a female candidate won by a small margin. The applications are endless.

RDD can also be useful in industry, but to a lesser extent. For example, suppose a bank offers a credit card to all its customers, but charges a fee to those with an account balance below 5,000 USD. This creates a discontinuity in the way the card is offered, where customers with balances above the threshold are more likely to choose the prime card, while

those with balances below the threshold are not. Thus, RDD can be applied to compare the effects of having a prime card versus a regular card, provided that customers above and below the threshold are similar in other respects.

Regarding the relevance of discontinuity designs in the industry, I believe that it is less applicable since firms could easily conduct experiments to randomize eligibility, as we have discussed earlier. However, let us suppose, for the sake of this example, that running such experiments would be time-consuming. Maybe because the required sample is too big, due to low compliance.

In contrast, the bank in question already has data following the discontinuity design described previously. Therefore, the bank can leverage this data to determine the effect of the prime credit card. How can the bank use the discontinuity for this purpose? The basic idea is to recognize that the threshold can be understood as an instrument since crossing it increases the likelihood of receiving the treatment.

In the following image, you can see how the discontinuity design relates to instrumental variables. The bottom part shows the counterfactual treatment by account balance. Since the instrument is crossing the 5,000 USD threshold, you can observe T_0 when *balance* < 5,000 and T_1 otherwise.

Moreover, since the instrument increases the chance of getting the treatment (the prime card), there is a jump in $P(T = 1)$ once you cross the threshold. The upper part of the

plot reflects how those changes in treatment probability impact the outcome.

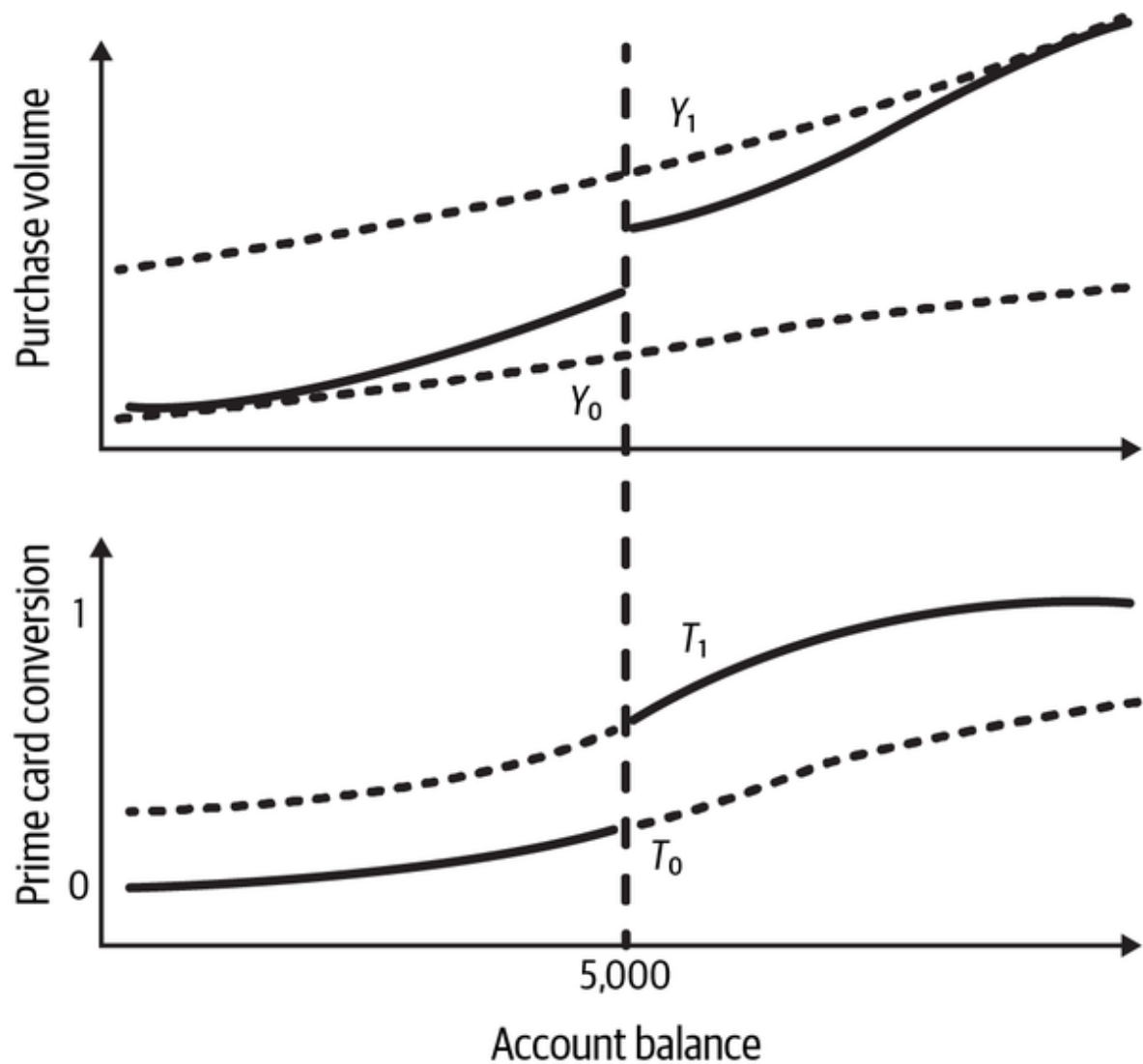


Figure 11-4. Potential outcomes and potential treatment in a discontinuity design

The probability of treatment being less than one, even above the threshold, makes the outcome you observe less than the true potential outcome Y_1 . By the same token, the outcome you observe below the threshold is higher than the true potential outcome Y_0 . This makes it look like the treatment

effect at the threshold is smaller than it actually is and you will have to use IV to correct for that.

Discontinuity Design Assumptions

Besides the IV assumptions, the discontinuity design requires one further assumption about the smoothness of the potential outcomes and potential treatment functions. Let's define a running variable R such that the treatment probability is a discontinuous function of that variable at a threshold $R = c$. In your banking example, R would be the account balance and $c = 5,000$.

Now, you need to assume that:

$$\begin{aligned}\lim_{r \rightarrow c^-} E[Y_t | R = r] &= \lim_{r \rightarrow c^+} E[Y_t | R = r] \\ \lim_{r \rightarrow c^-} E[T_z | R = r] &= \lim_{r \rightarrow c^+} E[T_z | R = r]\end{aligned}$$

In other words, the potential outcome Y_t and potential treatment T_z at the discontinuity $R = c$ are the same if you approach them from the left or from the right.

With those assumptions at hand, you can derive the local average treatment effect estimator for a discontinuity design:

$$\begin{aligned}LATE &= \frac{\lim_{r \rightarrow c^+} E[Y | R = r] - \lim_{r \rightarrow c^-} E[Y | R = r]}{\lim_{r \rightarrow c^+} E[T | R = r] - \lim_{r \rightarrow c^-} E[T | R = r]} \\ &= E[Y_1 - Y_0 | T_1 > T_0, R = c]\end{aligned}$$

Importantly, this estimator is local in two senses. First, it is local because it only gives the treatment effect at the threshold

$R = c$. This is the discontinuity design locality. Second, it is local because it only estimates the treatment effect for the compliers. This is the IV locality.

Intention to Treat Effect

In the top part of [Figure 11-4](#), the jump in the observed outcome at the threshold is the intention-to-treat effect, since it measures how the outcome changes as you change the instrument. Let's now see how you can estimate it, since this will be the numerator of your final IV estimate. To do so, let's first read the data containing information on the customer's account balance, whether or not they choose the prime card, and what their purchase volume is:

```
In [19]: df_dd = pd.read_csv("./data/prime_card_
        df_dd.head()
```

	balance	prime_card	pv
0	12100.0	1	356.472
1	4400.0	1	268.172
2	4600.0	1	668.896
3	3500.0	1	428.094
4	12700.0	1	1619.793

Next, you need to regress the outcome variable on the running variable R (balance) interacted with a dummy for being above the threshold ($R > c$):

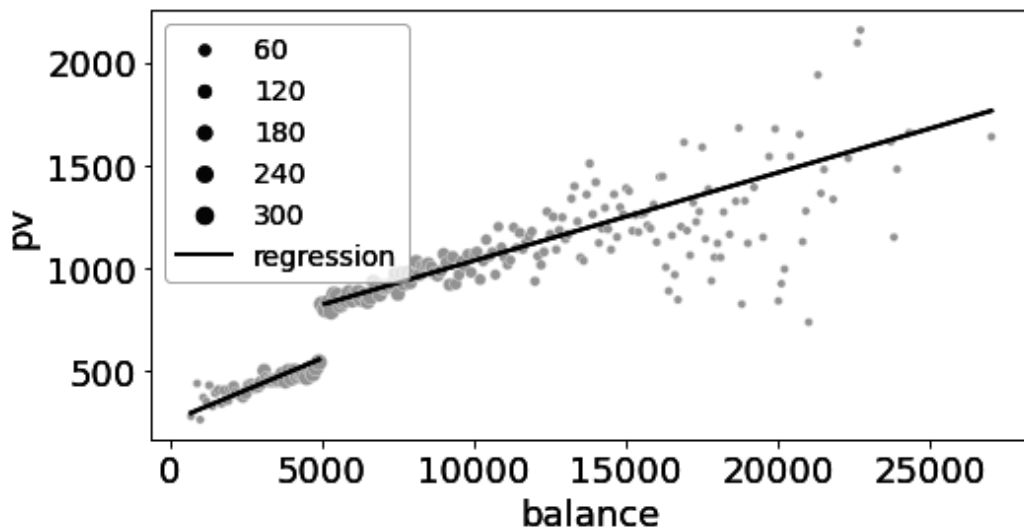
$$y_i = \beta_0 + \beta_1 r_i + \beta_2 \mathbb{1}(r_i > c) + \beta_3 \mathbb{1}(r_i > c) r_i$$

The parameter estimate associated with crossing the threshold, $\widehat{\beta}_2$, can be interpreted as the intention-to-treat effect:

```
In [20]: m = smf.ols(f"pv~balance*I(balance>5000)
           m.summary().tables[1]
```

	coef	std err	t	P> t
Intercept	251.1350	19.632	12.792	0.00
I(balance > 5000)	354.7539	22.992	15.430	0.00
[T.True]				
balance	0.0616	0.005	11.892	0.00
balance:I(balance	-0.0187	0.005	-3.488	0.00
> 5000)[T.True]				

Notice that this is essentially running two regression lines: one above and one below the threshold. If compliance was not an issue, meaning that everyone above the threshold would get the treatment and everyone below the threshold would get the control, you could still use this approach. If that were the case, compliance would be 100%, the ITTE would already be the ATE:



The IV Estimate

Since compliance is not 100%, you need to divide the intention-to-treat effect by the compliance rate. In the context of a discontinuity design, that would be how much the treatment probability changes as you cross the threshold. To estimate this number, you can simply repeat the previous procedure, replacing the outcome variable, `pv`, with the treatment, `prime_card`. Here is a simple function to compute the IV estimate in a discontinuity design. It estimates the ITTE and the compliance rate and divides one by the other:

```
In [21]: def rdd_iv(data, y, t, r, cutoff):
           compliance = smf.ols(f"{t}~{r}*I({r}>{cutoff})").fit().params[1]
           itte = smf.ols(f"{y}~{r}*I({r}>{cutoff})").fit().params[1]
           param = f"I({r} > {cutoff})[T.True]".fit().params[1]
```



```

        return itte.params[param]/compliance

rdd_iv(df_dd, y="pv", t="prime_card", :

```

```
Out[21]: 654.3917626767736
```

Applying this function to your data yields an estimate that is pretty close to the true LATE. Remember that you can check this since this dataset contains the individual level treatment effect stored in the `tau` column and the compliance category:

```

In [22]: (df_dd
          .round({"balance":-2}) # round to nearest
          .query("balance==5000 & categ==' compl:

```

```
Out[22]: 700.0
```

Finally, although you could derive a formula to calculate the confidence interval of that estimator, the easiest way is to simply wrap the entire function in a bootstrap procedure. I'll

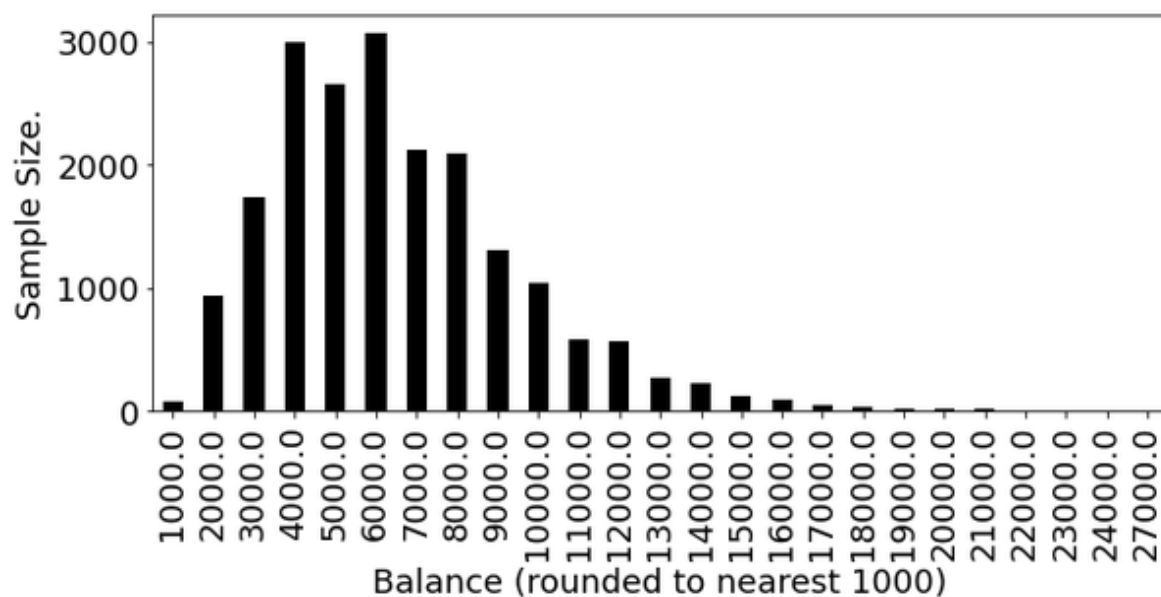
hide the code for this since it is fairly repetitive, but you can see the resulting interval here:

```
array([535.49935691, 781.24156232])
```

Bunching

Before closing this chapter, I just wanted to mention a potential issue to the discontinuity design identification. If the units (customers in your example) can manipulate the running variable, they can also self-select into the treatment group. In the prime credit card example, customers could decide to increase their deposits until it reached just 5,000 so that they would get the prime credit card for free. This would violate the assumption about the smoothness in the potential outcomes, since those just above the threshold would no longer be comparable to those just below it.

A simple and visual way to check if this is happening is to plot the density around the threshold. If units are self-selecting into the treatment, you would expect a huge spike in the density at the threshold. Fortunately, it does not appear to be the case with this data:



Key Ideas

In this chapter you learned that noncompliance becomes an issue when people can choose not to take a treatment. This is fairly common in the industry, as companies tend to have a pool of optional products or services. In those situations, the customer choice confounds the effect of the product or service, even if the company can randomize their availability.

You also learned about the compliance groups or types:

Compliers

Those who take the treatment that was assigned to them.

Always takers

Those who always take the treatment, regardless of the assignment.

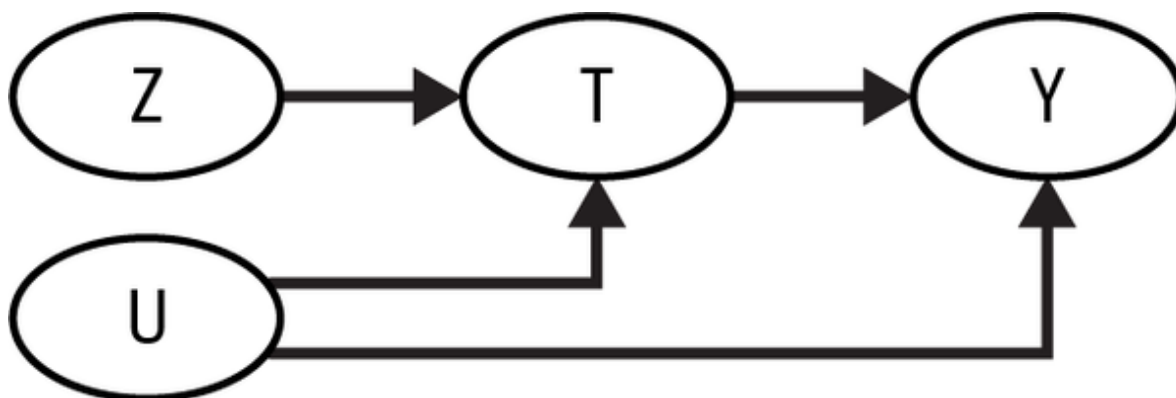
Never takers

Those who never take the treatment, regardless of the assignment.

Defiers

Those who take the opposite treatment from the one assigned.

And you learned how to use instruments to deal with noncompliance. Namely, an instrument Z is a variable that (1) impacts the treatment in a nonconfounded way and (2) doesn't impact the outcome, unless through the treatment:



On top of that, if you assume that the instrument flips the treatment in a single direction (monotonicity assumption), you can use it to identify the average treatment effect on the compliers:

$$E[Y_1 - Y_0 | T_1 > T_0] = \frac{E[Y|Z = 1] - E[Y|Z = 0]}{E[T|Z = 1] - E[T|Z = 0]}$$

In other words, all you have to do is to normalize the intention to treat the effect by the compliance rate, both of which are easy to identify if the instrument is randomized.

However, there is still a price to pay in terms of variance. If compliance is low, the variance of the instrumental variable estimate will be substantially larger than that of OLS.

Particularly, if compliance is 50%, you would need 4x more samples to achieve the same standard errors as if compliance was not an issue (100% compliance). There are some additional tricks to decrease the variance, but the most promising one seems to be finding variables that are good at predicting the outcome, pretty much as it was the case with OLS.

Additionally, you learned that discontinuities in your data could also be treated as instruments. In general, you probably won't need to rely on them, since deploying experiments is fairly common and easy in the industry. Still, in the case that experiments are not available, you can leverage those discontinuities to identify the local average treatment effect.

QUARTER OF BIRTH INSTRUMENT

As I said earlier, it is pretty hard to find valid instrumental variables in the wild, but quarter of birth might be one of those. In the US, being born in the last quarter means you'll probably have more school years, since you'll join school earlier in your life. If quarter of birth doesn't affect income (unless through schooling) and is as good as random, economists can use it to identify the effect of school on income.

By doing that, economists estimated that we should expect one extra year of education to increase wages by 8.5%, on average:

